

financial-db

台股研究級資料倉儲

從資料蒐集、分層建模到跨來源驗證

摘要

本專案以 Parquet 與 DuckDB 建置台股日資料倉儲，共 3,394 萬列，來源是 FinMind API 與 TEJ Smart Wizard (Excel 增益集) 兩條各自獨立的管線。兩條管線遵守同一套三層契約：raw 保留上游原貌，processed 是有型別、去重、分區的查詢契約，DuckDB 只存 view (共 39 個)，可以隨時刪掉重建。

資料正確性靠三件事維持：286 個自動化測試、63 項唯讀健康檢查，以及用第二個來源做交叉驗證。在兩個來源共有的 2,188 檔證券上，868 萬筆日報酬中有 99.96% 相差在 10bp 以內；其餘 10,008 個不一致的交易日，每一筆都歸因到具體原因。

主要發現有三項。第一，FinMind 的還原價在上市股 2003 年以前、上櫃股 2008 年以前並沒有還原除權息。第二，同一套比對抓到本專案自己管線裡的一個去重錯誤，已修正並補上測試。第三，FinMind 的未還原價另有無成交日記成 0、早期成交量單位不一致等缺陷。結論是：計算報酬與回測以 TEJ 為準，全市場廣度與 2000 年以前的歷史用 FinMind 的未還原價。

目錄

1	專案動機與目標.....	4
2	整體架構.....	4
2.1	三層契約.....	4
2.2	為什麼選擇 Parquet 與 DuckDB.....	5
3	資料內容.....	6
3.1	資料集總覽.....	6
3.2	兩個來源的定位.....	7
4	資料蒐集與建置.....	7
4.1	FinMind : API 管線.....	7
4.2	TEJ : 以 Excel 為來源的管線.....	8
5	資料處理與 Schema.....	9
5.1	raw 到 processed 的處理.....	9
5.2	FinMind 的三層 Schema.....	9
5.3	TEJ 的三層 Schema.....	10
5.4	view 層.....	11
6	更新機制與資料品質.....	13
6.1	排程.....	13
6.2	自我修復與失敗處理.....	13
6.3	測試、健康檢查與已處理的上游缺陷.....	14
7	跨來源驗證 : FinMind 與 TEJ.....	14
7.1	涵蓋率與存活者偏誤.....	15
7.2	驗證方法.....	15
7.3	結果總覽.....	16

7.4	發現一：FinMind 的還原價，早期並沒有還原	16
7.5	發現二：交叉驗證抓到自己管線的錯誤	18
7.6	發現三：其他只有對照才看得到的缺陷	19
7.7	誰比較可信	20
8	各來源不可信之處與使用建議	21
9	限制與未來工作	22
10	結語	22
附錄	重現方式與檔案位置	23

1 專案動機與目標

做量化研究時，最容易被忽略的風險不在模型，而在資料。直接呼叫公開 API 取得的股價可以很快跑出回測結果，但這樣的結果常常好得不真實，原因多半出在三個地方。

- **存活者偏誤**。證券主檔只列出現在還在交易的股票。已經下市的公司從歷史橫斷面中消失，選股策略的報酬因此被高估。
- **還原價會被改寫**。每一次除權息都會重寫該股票整段還原歷史。只會往後追加資料的資料庫，舊資料會在不知不覺中過期。
- **單一來源無法自證**。單位錯誤、缺漏、沒有調整到的除權息，這些來源自己的錯，只看這一個來源是看不出來的。

本專案的目標，是建立一個能支撐回測研究的台股日資料倉儲，並且讓「資料是否正確」成為可以檢驗、可以重現的問題。具體要求如下：

- 每一層資料都能由上一層重建，資料只由程式產生，不手動修改。
- 保留已下市公司，讓歷史橫斷面不受存活者偏誤影響。
- 文件與程式斷言中的數字一律來自實測。
- 引入第二個獨立來源，逐日交叉驗證，並量化兩者的差異。

2 整體架構

2.1 三層契約

倉儲由兩條互相獨立的管線組成，分別對應 FinMind 與 TEJ 兩個來源。兩條管線刻意採用相同的分層方式，因此只需要一個心智模型就能理解兩邊（圖 1）。

- **raw 層**：來源給什麼就存什麼。FinMind 的欄名與日期字串原樣保留；TEJ 的 xlsx 活頁簿原封不動。這一層是重建的起點。
- **processed 層**：有正確型別、去重、依日期分區的 Parquet。這一層才是查詢契約，所有查詢只應該碰這一層。
- **view 層**：DuckDB 檔案裡一張資料表都沒有，只有蓋在 Parquet 上的 view 與 macro，檔案大小 268 KB，刪掉後重跑一支程式就能重建。

兩條管線的差異都來自來源本身的性質，而不是設計上的漂移。最重要的一點是：FinMind 的 raw 可以隨時向 API 重抓，TEJ 的 xlsx 則是授權資料，查詢定義存在檔案內部，損壞就無法重製，因此更新前一律先備份。

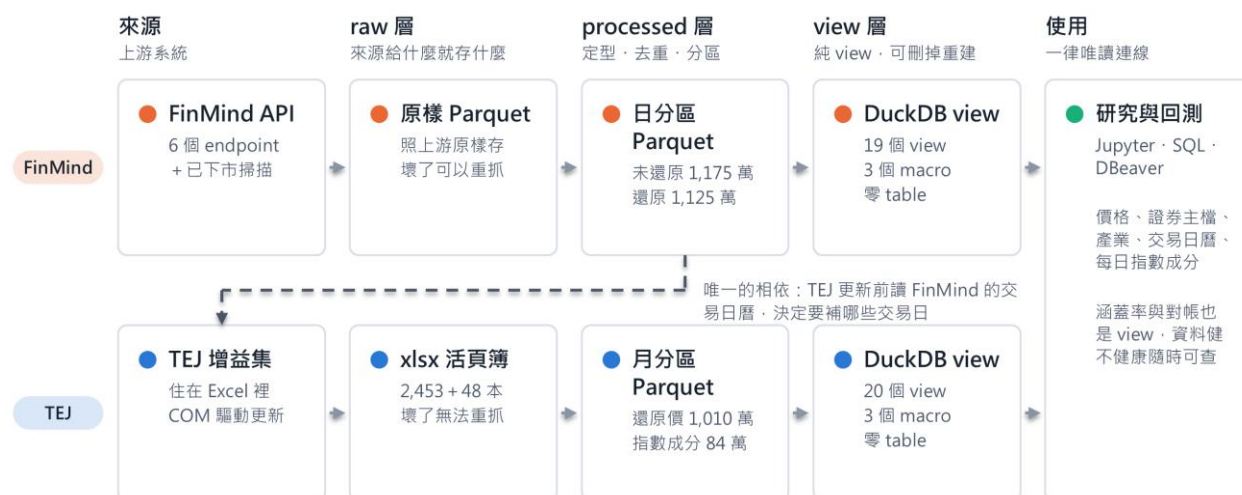


圖 1 整體架構。兩條管線各自經過 raw、processed、view 三層；唯一的相依是 TEJ 更新前要讀 FinMind 的交易日曆。圖中數字為列數。

2.2 為什麼選擇 Parquet 與 DuckDB

這個組合的出發點是讓 Parquet 成為唯一真相，資料庫本身不持有資料。這樣做有三個好處：任何工具都能直接讀取資料；view 層壞了可以直接丟掉重建；排程工作只需要在重新定義 view 的瞬間持有 DuckDB 的寫入鎖。各項設計取舍都以實測數字決定（表 1）。

表 1 主要設計取舍與實測結果

設計決定	理由	實測
DuckDB 只存 view	Parquet 是唯一真相；view 層可拋棄重建，也避開單一寫入者的檔案鎖	兩個 .duckdb 檔各 268 KB
FinMind 一天一個檔	每日更新只新增一個檔，寫入天然冪等；一天要嘛完整存在、要嘛完全不存在，對帳變成日期集合的比較	8,069 個日檔
TEJ 一月一個檔	TEJ 每次全量重建，不需要冪等追加；小檔案無法有效壓縮	全表掃描 624 ms → 41 ms，檔案 188.9 MB → 67.1 MB
日期查詢走 macro	分區鍵是 year / month，只寫日期條件不會裁剪檔案；macro 自動補上 year 條件	查一季資料 0.56 秒 → 0.13 秒

3 資料內容

3.1 資料集總覽

倉儲目前共有 3,394 萬列日資料 (表 2) 。FinMind 一側涵蓋全市場，TEJ 一側涵蓋上市櫃公司池與兩個指數的每日成分。

表 2 資料集總覽 (2026-10-04 實測)

資料集	來源	列數	證券數	期間
未還原日行情 (開高低收、成交量值、筆數)	FinMind	11,751,428	3,614	1994-10-01 → 2026-09-16
還原日行情	FinMind	11,252,462	3,154	1994-10-01 → 2026-09-16
還原日行情 (40 欄：估值、權重、處置旗標)	TEJ	10,101,676	2,453	2000-09-04 → 2026-10-02
指數每日成分 (TWN50、TM100)	TEJ	835,348	367	2002-07-05 → 2026-10-05
維度：證券主檔、產業鏈、交易日曆、已下市清單	FinMind	3,916 檔證券 8,141 個交易日	—	1994-10-01 起

FinMind 的價格範圍刻意排除權證與 TDR：全市場一天約回傳 4.4 萬檔，其中約 4 萬檔是權證，只保留普通股與 ETF 約 2,800 檔。目前沒有納入財報、三大法人、融資券、股利明細與盤中資料。

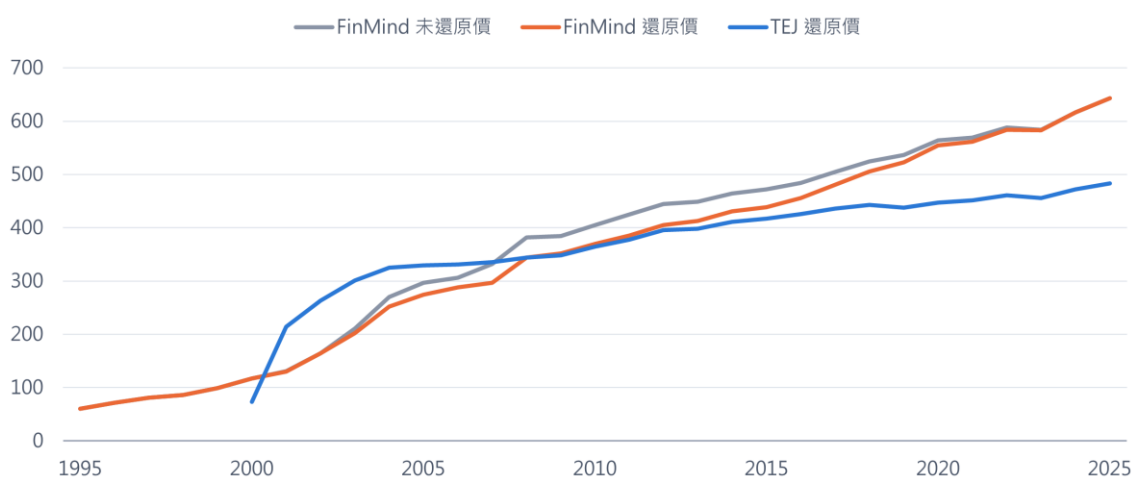


圖 2 各資料集每年的列數 (千列) 。未滿一年的 2026 年不列入。

3.2 兩個來源的定位

兩個來源並非重複，而是各有所長（表 3）。需要全市場橫斷面或 2000 年以前的歷史時用 FinMind；需要無偏誤的還原報酬、估值欄位或指數的歷史成分時用 TEJ。

表 3 FinMind 與 TEJ 的定位

面向	FinMind	TEJ
來源與取得	FinMind API v4 (HTTP + token)，隨時可以重抓	TEJ Smart Wizard Excel 增益集；授權資料，raw 無法重製
涵蓋	全市場 3,916 檔：上市、上櫃、興櫃、ETF、指數	公司池 2,453 檔，加上 TWN50 / TM100 每日成分
期間	1994-10-01 起	2000-09-04 起
已下市公司的還原價	找回的 767 檔中只有 7 檔有 (上游缺口)	462 檔已停止交易者全數保留完整歷史
每列欄位	精簡：開高低收、成交量值、筆數	40 欄：估值、殖利率、市值比重、漲跌停與處置旗標

4 資料蒐集與建置

4.1 FinMind：API 管線

FinMind 管線抓取 6 個 endpoint，再加上一個自行設計的全市場掃描。各步驟的順序不能調換（圖 3）：維度要先於價格，因為過濾範圍要用最新的證券名單；價格要先於交易日曆，因為早期日曆要從價格日期反推。



圖 3 FinMind 管線的處理順序。

- **兩種抓取形狀。**回補歷史時，一次呼叫取得一檔證券的全部歷史，約 3,600 次呼叫即可補完 32 年；每日更新時，一次呼叫取得全市場一天。從零建置整個資料庫約需 7,830 次呼叫、85 至 95 分鐘。

- **跨行程限流**。自行實作 token-bucket 限流器，狀態存在檔案裡，排程與手動執行共用同一份配額（方案上限的 90%）。配額用盡與伺服器錯誤各有退避重試，單次拒絕不會中斷整個回補。
- **找回已下市證券**。證券主檔只列現存證券，照著主檔回補就永遠不會知道已下市的公司存在過。做法是自 2004 年起每月抽一個交易日做全市場呼叫（271 次），記下所有出現過的代碼，再與下市公告聯集。兩個來源缺一不可：只靠掃描找到 296 檔、只靠公告 320 檔、兩者都有 151 檔，合計 767 檔。
- **原子寫入**。所有檔案都先寫到暫存檔再換名；全量重建則先寫到暫存目錄，完成後整個目錄換入，過程中原有資料始終完整可查。

4.2 TEJ：以 Excel 為來源的管線

TEJ 沒有提供 HTTP API，唯一能更新資料的是住在 Excel 裡的 TEJ Smart Wizard 增益集。因此這條管線的核心問題是：如何讓一個必須由人操作 Excel 的流程，變成可以排程、可以驗證的資料來源（圖 4）。



圖 4 TEJ 每日更新的流程。新增的日期如果沒有收盤價，就不存檔並中止整次更新。

- **以 COM 驅動隱藏的 Excel**。程式啟動一個獨立的 Excel 行程，自行載入增益集，呼叫同步版本的更新巨集，並以看門狗執行緒在逾時後結束行程。
- **視窗隔離**。增益集更新時會把 Excel 視窗翻成可見並搶走焦點。把執行緒綁到獨立的 Windows 桌面後，使用者桌面上的視窗數由 4 個變成 0 個，耗時 12.4 秒對 13.0 秒，沒有額外成本。
- **先補日期，再更新**。增益集不會自己產生新的日期列。程式先讀 FinMind 的交易日曆，找出每本活頁簿缺少的交易日，插入代碼與日期後才更新。
- **驗證後才存檔**。新增列的收盤價必須有值，否則不存檔並中止整次更新。raw 是不可重製的唯一真相，寧可停下也不寫進半筆資料。
- **從樣板建立新活頁簿**。查詢定義存在儲存格註解裡，而且不含證券代碼。複製既有活頁簿、改寫代碼欄再更新，就得到另一檔證券的完整歷史。兩千三百多本活頁簿由此自動建立；與手動下載的檔案逐格比對，230,796 格全部相同。

解析時不開 Excel，也不用一般的試算表函式庫，而是把 xlsx 當成壓縮檔，以串流方式讀取其中的 XML。表頭必須逐字符合契約，否則直接報錯，避免把另一種查詢的活頁簿混進同一份資料。

5 資料處理與 Schema

5.1 raw 到 processed 的處理

以 FinMind 的價格資料為例，處理分成四步：

- **改名與定型**。上游的 max、min 與 SQL 聚合函式同名，改為 high、low；日期字串轉為 DATE；金額轉為 BIGINT（單日成交值會超過 32 位元整數）。證券代碼刻意保留字串，因為 0050、00679B 轉成數字就壞了。
- **過濾範圍**。全市場一天回傳 43,727 檔，依代碼形狀只保留普通股與 ETF 共 2,821 檔。
- **重新分區**。raw 是一檔一個檔案，processed 是一天一個檔案。約 1,200 萬列的重新分區交給 DuckDB 逐年分塊處理。
- **去重**。每張表都先寫下自然鍵。同一個（證券，日期）可能同時來自兩種抓取形狀：未還原價以全市場日檔為準，還原價以單檔全歷史為準（原因見 7.5 節）。

維度表的去重是一個具體的教訓。上游每次重爬都會把日期欄重新蓋章，沒有鍵概念的去重（SELECT DISTINCT *）會把同一筆事實每爬一次多存一列；產業表在第二次抓取後由 6,871 列變成 11,520 列。改以（證券，產業，子產業）為鍵之後回到 6,871 列，並在健康檢查中加上守門規則。TEJ 一側則不去重，直接斷言鍵唯一，違反就讓重建失敗。

5.2 FinMind 的三層 Schema



圖 5 FinMind 價格資料在三層中的欄位。

表 4 FinMind 價格資料：raw 與 processed 的欄位對照

raw 欄位	raw 型別	processed 欄位	processed 型別	說明
date	字串	date	DATE	鍵
stock_id	字串	stock_id	VARCHAR	鍵；前導零有意義
open、close、spread	double	同名	DOUBLE	
max、min	double	high、low	DOUBLE	避開 SQL 聚合函式名
Trading_Volume	int64	volume	BIGINT	單位是股
Trading_money	int64	turnover_value	BIGINT	單位是元
Trading_turnover	int64	transactions	BIGINT	成交筆數
—	—	source	VARCHAR	market 或 ticker
—	—	ingested_at	TIMESTAMPTZ	寫入時間
—	—	year、month	分區鍵	目錄層級

維度表有證券主檔（依內容去重，日期欄代表首次出現的日子）、產業鏈（多對多，一檔股票對應 1 至 61 列）、交易日曆（官方日曆加上由價格反推的 1,204 天）、已下市清單（掃描與公告的聯集減去現有主檔）。

5.3 TEJ 的三層 Schema



圖 6 TEJ 還原價資料在三層中的欄位。

TEJ 每本活頁簿固定 35 欄，表頭為中文。處理時把「證券代碼」欄切成代碼與名稱、把 Excel 序號轉成日期、把中文欄名轉成英文，再加上來源檔名與寫入時間，成為 38 欄；view 再加上兩個分區鍵，合計 40 欄（表 5）。

表 5 TEJ 還原日行情的欄位分組 (processed 層 38 欄)

分組	欄位	說明
識別	stock_id, stock_name, date	鍵為 (stock_id, date) , 斷言唯一
還原價	open, high, low, close	回溯調整後的價格
成交	volume_k_shares, turnover_k, transactions, turnover_pct	千股、千元、筆數、週轉率
報酬	return_pct, return_ln, price_change, high_low_spread_pct	
規模與比重	shares_outstanding_k, market_cap_m, market_cap_weight_pct, turnover_weight_pct	千股、百萬元
估值	pe_tse, pe_tej, pbr_tse, pbr_tej, psr_tej, div_yield_tse, cash_div_yield	
未還原報價	bid, offer, next_ref_price, next_limit_up, next_limit_down	原始報價，與還原價不同尺度
旗標	limit_flag, attention_flag, disposition_flag, full_delivery_flag, market	漲跌停、注意、處置、全額交割、市場別
來源	source_file, ingested_at	

這張表有一個重要的陷阱：同一列只有開高低收是還原價，買賣報價與次日參考價是未還原的原始報價。以 2330 在 2000-09-04 為例，還原收盤價是 27.99，次日參考價是 133.50，相差 4.8 倍，兩者不能混用。反過來說，未還原欄位永遠不會被改寫，所以測試中的固定值只用這些欄位。

指數成分資料 (每列是「指數 × 交易日 × 成分股」) 有 14 欄，包含指數因子、公眾流通係數、股數、前一日還原收盤價與前一日市值比重。它的日期是生效日，資料來自前一個交易日，因此比價格資料領先一個交易日。每日成分數並不固定：TWN50 為 46 至 51 檔，TM100 為 98 至 102 檔。

5.4 view 層

兩個資料庫共提供 39 個 view 與 6 個 macro (表 6)。view 層的作用不只是方便查詢，也把「怎麼查才對」寫進資料庫：回測用的證券範圍、分區裁剪、涵蓋率與對帳，都有對應的 view。

表 6 view 的分組

用途	FinMind (19 view + 3 macro)	TEJ (20 view + 3 macro)
價格	v_daily_prices, v_adj_daily_prices, v_prices_combined, v_daily_prices_enriched	v_adj_daily_prices, v_securities
回測用的證券範圍	v_securities_all, v_delisted_securities, v_stock_info_latest, v_stock_info_history, v_securities, v_indices	v_index_constituents, v_index_membership, v_index_universe
產業與日曆	v_industry, v_industry_by_stock, v_trading_calendar, v_last_trading_day	v_trading_calendar, v_last_trading_day, v_index_calendar, v_index_last_trading_day
涵蓋率與對帳	v_daily_coverage, v_missing_trading_days, v_thin_trading_days, v_known_empty_days	v_daily_coverage, v_thin_trading_days, v_security_gaps, v_workbooks, v_workbook_status, v_index_daily_coverage, v_thin_index_days, v_index_price_gaps, v_index_workbooks, v_index_workbook_status
維運	v_ingestion_runs	v_ingestion_runs
macro	daily_prices_between, adj_daily_prices_between, prices_combined_between	adj_daily_prices_between, index_constituents_between, index_members_on

查詢一律以唯讀方式連線。寫入連線會鎖住 DuckDB 檔案，擋掉排程更新；範例如下：

```
import duckdb
con = duckdb.connect(r"...db\finmind\finmind.duckdb", read_only=True)
con.execute("""
    SELECT date, stock_id, close, volume
    FROM daily_prices_between('2024-01-01', '2024-03-31')  -- macro 會裁剪分區
    WHERE stock_id = '2330'
    """).df()
```

6 更新機制與資料品質

6.1 排程

更新由 Windows 工作排程器執行，分成每日增量與每週全量兩種（圖 7、表 7）。TEJ 的時間刻意排在 FinMind 之後，因為它要讀 FinMind 的交易日曆，而 FinMind 更新時會重寫這份日曆。



圖 7 一週的排程。

表 7 排程工作

工作	時間	內容	成本
FinMind 每日更新	週一至週六 18:30	3 個維度快照，加上缺漏日的全市場呼叫，重建 view	9 至 22 次呼叫，秒級
FinMind 每週重抓	週日 02:00	還原價全量重抓，並做四類對帳	約 3,200 次呼叫，約 45 分
TEJ 每日更新	每日 18:45	指數檔與當期成分約 150 本補日，重建並驗證	約 55 分 (排程尚未註冊)
TEJ 每週全量	週日 03:00	指數檔與全部 2,453 本活頁簿重抓	約 7 小時

6.2 自我修復與失敗處理

- 每日更新抓三個集合的聯集**。新的日期、最近 3 個交易日 (上游在收盤後會修正當日成交量，例如 0050 某日由 39,946,000 股修正為 43,382,378 股)，以及範圍內的缺漏日。漏跑的日子在下一次執行時自動補齊；超過 30 天則拒絕執行，要求改跑回補。
- 還原價每週全量重抓**。除權息會改寫整段歷史，日期補齊不等於數字正確，所以不能只追加。

- **每週對帳**。檢查四類問題：日曆上有但資料裡沒有的日子、列數異常偏低的日子、完全沒有資料的現存證券、長期沒有更新的現存證券。
- **失敗就停**。任何一個階段失敗，後面的階段就不執行。TEJ 更新失敗時刻意不重建，因為用舊的活頁簿重建只會得到一樣的資料庫，卻讓這次執行看起來成功。
- **每次執行都留下紀錄**。執行結果寫成 JSONL，並以 view (v_ingestion_runs) 提供查詢，包含狀態、耗時、指標與錯誤訊息。

6.3 測試、健康檢查與已處理的上游缺陷

兩條管線共有 286 個自動化測試 (FinMind 210 個、TEJ 76 個) 與 63 項唯讀健康檢查 (28 項與 35 項)。健康檢查涵蓋鍵的唯一性、空值、日期範圍、固定值比對、分區鍵與日期是否一致等，排程結束後自動執行，失敗時以非零代碼結束。上游資料本身的缺陷，每一項都有對應的處理與測試 (表 8)。

表 8 處理過的上游缺陷

缺陷	證據	處理方式
日期欄是字串「None」	證券主檔 32 列	轉成真正的 NULL，否則轉型會直接失敗
1999 年以前有週六盤	由價格反推的 1,204 天中有 176 個週六	不寫任何「週末必無交易」的邏輯
官方日曆列了實際休市的日子	2026-07-10：日曆有，全市場回傳 0 檔	向 API 確認一次後記錄，不再當成缺漏反覆重抓
收盤後成交量被修正	0050：39,946,000 → 43,382,378 股	每日重抓最近 3 個交易日
產業名稱飄移	創新版與創新版等	別名表正規化，原值另存
成交值超過 32 位元整數	最大 52,822,527,292	計數欄位一律 BIGINT

7 跨來源驗證：FinMind 與 TEJ

有了兩個獨立來源之後，就可以回答單一來源無法回答的問題：兩者哪裡不同？誰是對的？本章的所有數字都由分析程式產生，可以重跑 (見附錄)。

7.1 涵蓋率與存活者偏誤

以 TEJ 的上市櫃資料列為基準，逐列檢查 FinMind 是否也有同一檔、同一天的資料（圖 8）。

FinMind 的未還原價自 2004 年起涵蓋率接近 100%，更早的年份只有現存公司；2007 年的下凹來自該年 1 至 4 月上櫃股的資料缺漏。還原價的涵蓋率更低，因為上游幾乎不提供已下市公司的還原價。

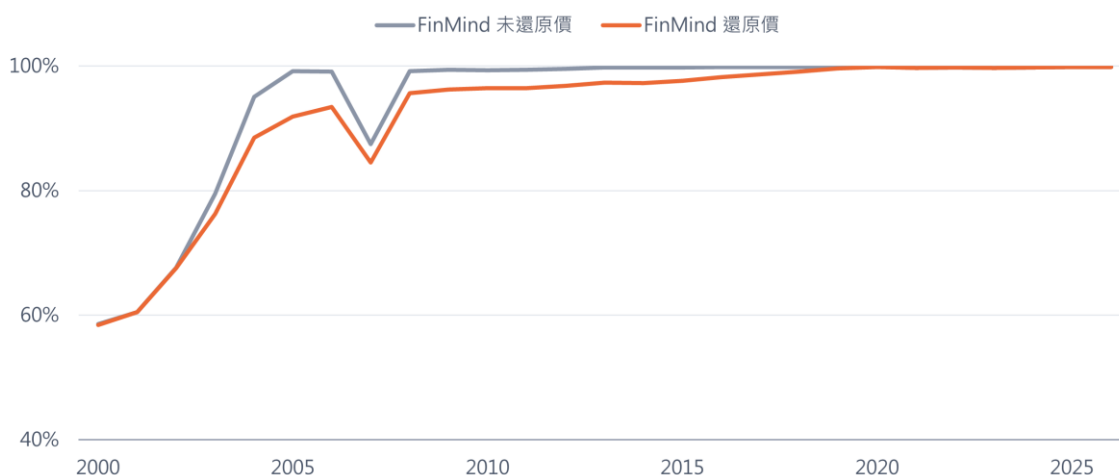


圖 8 TEJ 上市櫃資料列中，FinMind 也有的比例（逐年）。

- FinMind 找回的 767 檔已下市證券中，466 檔有未還原價，只有 7 檔有還原價。
- TEJ 的 2,453 檔中有 462 檔已停止交易，全數保有完整還原歷史；FinMind 的還原價只涵蓋其中 197 檔。
- FinMind 能做無偏誤橫斷面的最早日期是 2004-02-11，也就是全市場端點的資料起點。

7.2 驗證方法

比較的對象是日報酬，而不是價格水準。兩個來源都把歷史回溯調整到最新基準，只要兩邊重抓的日期不同，整段價格水準就會相差一個常數。逐列比較還原收盤價，只有 69.3% 相差在 0.01 元以內，但這並不代表有錯。同一檔、同一天的報酬則不受基準影響。

- 比對範圍：兩邊共有的 2,188 檔證券、941 萬列（2000-09-04 → 2026-09-16）。
- 乾淨樣本：上市櫃、且前後兩天都有成交的日報酬，共 868 萬筆。
- 第三把尺：以 FinMind 的未還原報酬當基準。某一天誰的報酬等於未還原報酬，誰那天就沒有做調整。
- 合理性檢驗：台股有漲跌幅限制（2015 年 6 月以前 7%，之後 10%）。還原後仍超過限制的報酬，不是真實事件，就是沒有調整乾淨。

7.3 結果總覽

在乾淨樣本中，99.96% 的日報酬兩邊相差在 10bp 以內。上市櫃股票中，兩邊相差超過 1% 的交易日共 10,008 個，每一筆都可以歸到下列原因之一（圖 9、表 9）。

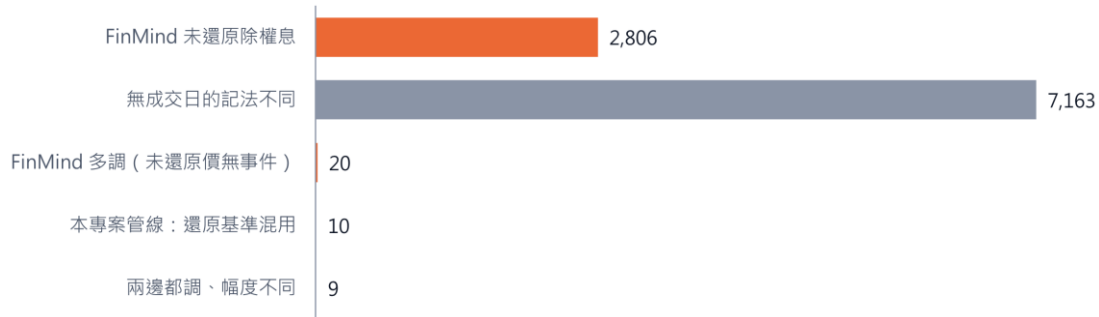


圖 9 兩邊日報酬相差超過 1% 的交易日數，依原因分類（上市櫃）。

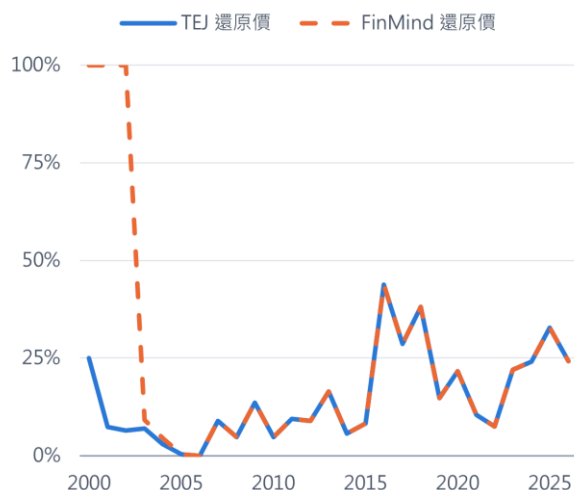
表 9 差異的原因

原因	交易日數	證券數	判定方式
無成交日的記法不同	7,163	401	FinMind 未還原收盤價為 0 的日子及其次日
FinMind 未還原除權息	2,806	864	FinMind 的報酬等於未還原報酬，TEJ 不等於
FinMind 多調	20	20	TEJ 的報酬等於未還原報酬，FinMind 不等於
本專案管線：還原基準混用	10	10	落在最近一次每週重抓之後的日子
兩邊都調、幅度不同	9	9	兩邊都不等於未還原報酬

7.4 發現一：FinMind 的還原價，早期並沒有還原

「FinMind 未還原除權息」的日子幾乎全部落在 2007 年以前。為了確認，逐年統計未還原價中超過漲跌幅限制的跳空（大致就是除權息日），再看還原之後還剩下多少（圖 10、表 10）。

上市



上櫃

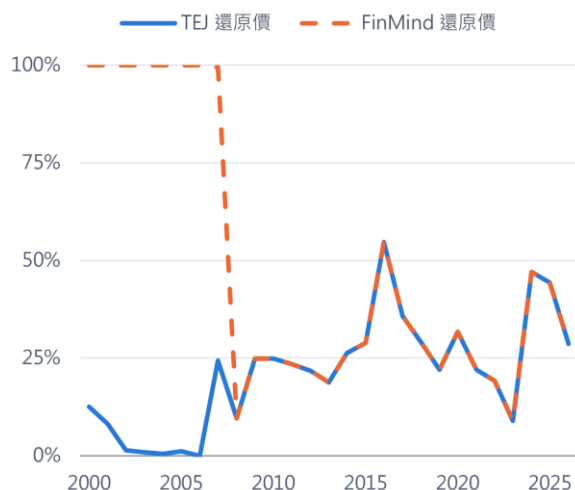


圖 10 每年「未還原價超過漲跌幅」的跳空中，還原後仍然留著的比例。100% 表示完全沒有調整；兩條線重合表示兩個來源一致。

表 10 未還原價超過漲跌幅的跳空數，以及還原後仍殘留的數量（2000–2009）

年	上市 未還原跳空	上市 FinMind 殘留	上市 TEJ 殘留	上櫃 未還原跳空	上櫃 FinMind 殘留	上櫃 TEJ 殘留
2000	16	16	4	8	8	1
2001	191	191	14	74	74	6
2002	157	157	10	70	70	1
2003	174	16	12	107	107	1
2004	245	11	7	187	187	1
2005	257	1	1	187	187	2
2006	221	0	0	192	192	0
2007	180	16	16	201	200	49
2008	319	15	15	249	24	24
2009	140	19	19	137	34	34

結論：上市股到 2002 年、上櫃股到 2007 年，FinMind 還原價中的跳空與未還原價完全相同，也就是沒有調整。共影響 2,806 個交易日、864 檔。拿這段期間的 FinMind 還原價做回測，除權息會被當成單日暴跌。2008 年以後兩個來源一致：2,074 檔中有 1,935 檔，整段累積報酬相差不到 1%。

7.5 發現二：交叉驗證抓到自己管線的錯誤

比對過程中有一類差異很特別：FinMind 自己的單檔歷史與 TEJ 一致，但本專案 processed 層的數字卻不同。以 6669 緯穎為例，它在 2026-09-02 除權，修正前的 processed 層在 8 月 21 日與 8 月 31 日分別出現 +195% 與 -67% 的假報酬（圖 11）。

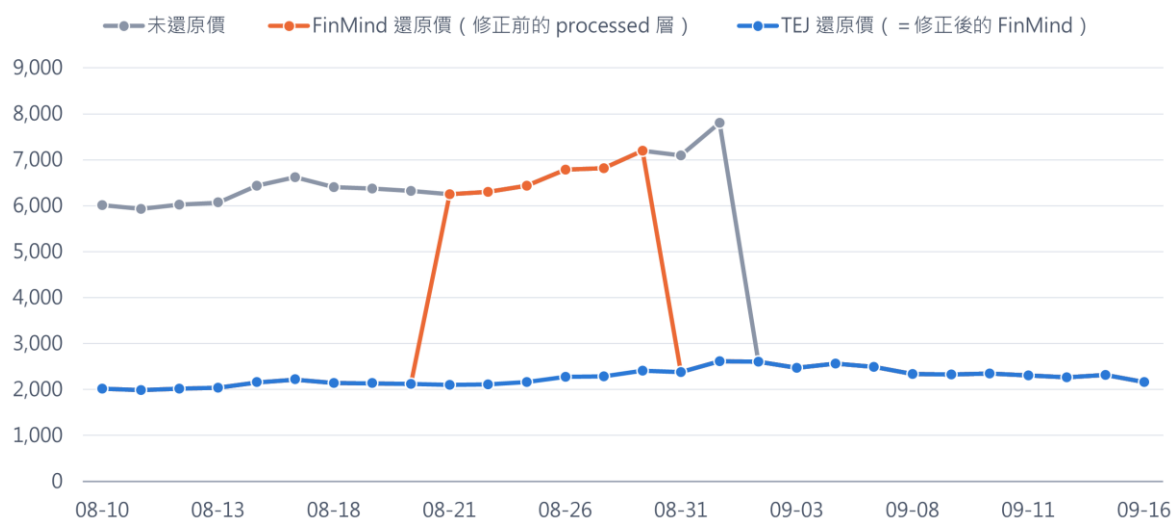


圖 11 6669 緯穎在除權日前後的三條序列。橘線是修正前的 processed 層；修正後與藍線重合。

原因。同一個（證券，日期）可能同時存在於「單檔全歷史」與「全市場日檔」兩種 raw 檔案。原本的去重規則一律讓全市場日檔優先，理由是它是收盤後的結算快照。這對未還原價是對的，對還原價卻是錯的：還原價的每個值只在抓取當下的還原基準下有意義，全市場日檔只是某一天在當時基準下的快照。每週重抓了正確的全歷史之後，重建時又被舊基準的日檔蓋回去，所以這個錯誤每週重抓也修不好。

修正。還原價改為單檔全歷史優先，全市場日檔只補上全歷史還沒涵蓋的日子；去重優先序改為每個資料集各自設定。同時新增 4 個測試與 1 條健康檢查（還原價中來自日檔的交易日數超過 7 天就警告），並從 raw 重建，過程不需要呼叫 API。

表 11 修正前後的比較

指標	修正前	修正後
受舊基準影響的資料	319 檔、1,046 列	0
還原價中來自日檔的交易日數	19 天	6 天
來自本專案管線的不一致交易日	105 天	10 天
超過漲跌幅限制的還原報酬（FinMind）	2,156	2,141

修正後剩下的 10 天，都落在最近一次每週重抓之後：這些日子只有日檔，如果期間有除權息，就會與之前的歷史處於不同基準，要等下一次每週重抓才會一致。這是還原價不能只追加的本質限制，窗口最長一週，健康檢查會在窗口過長時警告。

7.6 發現三：其他只有對照才看得到的缺陷

表 12 FinMind 一側的其他缺陷

現象	規模（實測）	影響
無成交日的收盤價記成 0	未還原價 387,319 列（3.3%），其中 36,489 列仍有成交量	直接計算報酬會得到 -100%
成交量單位不一致	2000 至 2003 年有 15% 至 27% 的列，成交量恰為 TEJ 的千分之一	早期成交量不能直接跨年比較
上櫃資料缺漏	2007 年 1 至 4 月，上櫃股幾乎整段沒有資料	該期間的橫斷面不完整
減資後復牌未調整	818 個停牌後跳空超過 25% 的事件：TEJ 消除 662 個，FinMind 消除 405 個	FinMind 有 413 個完全沒有調整
多餘的調整	20 個交易日：未還原價沒有事件，還原價卻跳動（例如 6949 在 2024-03-11 為 -95%）	屬上游錯誤

其中無成交日的處理方式值得多說明。FinMind 把沒有成交的日子記成 0，還原價則沿用前一日收盤價；TEJ 記的是參考價。在 7,163 個因此不一致的交易日中，有 1,614 天 TEJ 的報酬正好落在漲跌停價，與「無成交但有漲跌停申報時以漲跌停價為收盤價」的情形相符；這個解釋尚未逐筆向交易所資料查證。FinMind 則要等到下一次成交才一次反映，累積報酬最後相同，但日報酬的時間點不同。

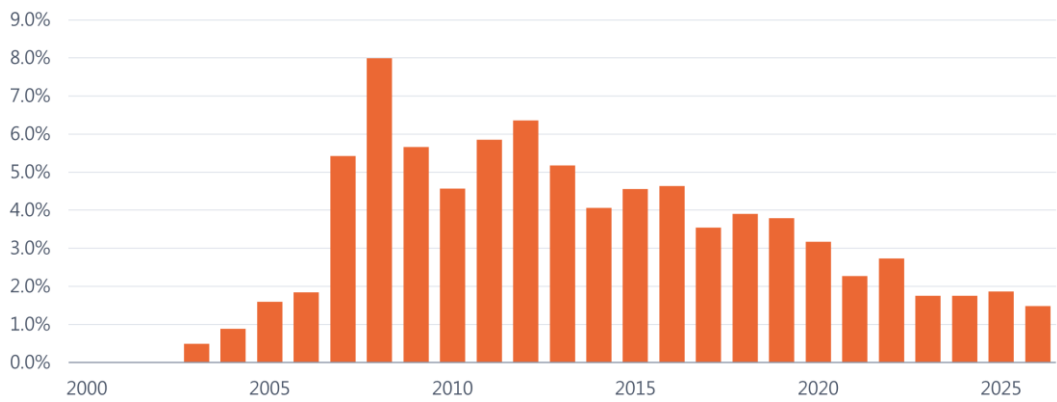


圖 12 FinMind 未還原價中，收盤價記成 0 的列所占比例（逐年）。

7.7 誰比較可信

最後用漲跌幅限制做總檢驗。在同一批 867 萬筆日報酬中，未還原價有 6,069 筆超過當時的漲跌幅限制；還原之後，FinMind 剩 2,141 筆，TEJ 剩 851 筆（圖 13）。TEJ 剩下的多數是真實事件，例如新掛牌前五個交易日沒有漲跌幅限制。

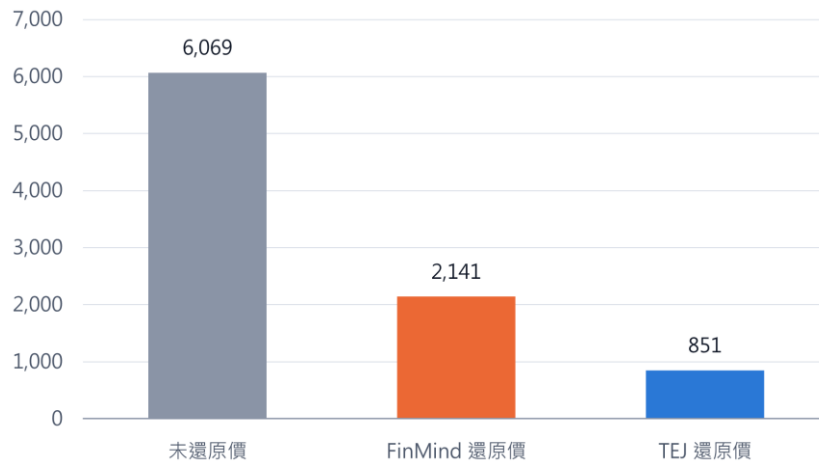


圖 13 同一批日報酬中，超過當時漲跌幅限制的筆數。

以整段累積報酬來看，2,101 檔中有 775 檔，兩個來源相差超過 10%，來源就是 2008 年以前沒有還原的除權息。因此就「用於計算報酬與回測的還原價」這個用途而言，TEJ 比較可信；這個結論不代表 TEJ 的每一筆資料都正確，下一章列出它的疑點。

8 各來源不可信之處與使用建議

表 13 兩個來源各自不能直接相信的地方

來源	不可信之處
FinMind 還原價	上市股 2003 年以前、上櫃股 2008 年以前沒有還原除權息；減資復牌多數未調整；已下市公司幾乎沒有還原價
FinMind 未還原價	無成交日的價格記成 0；2000 至 2003 年成交量單位不一致；2007 年 1 至 4 月上櫃缺漏；2004-02-11 以前只有現存公司
FinMind 維度	官方交易日曆含實際休市的日子；每次重爬都會改日期戳記；產業分類沒有歷史版本；下市公告只涵蓋約三分之一的已下市證券
TEJ 還原價	同一列混有還原與未還原兩種尺度；18 個交易日的還原報酬超過漲跌幅，而未還原價與 FinMind 都沒有，原因待查；內建報酬率欄位與由還原收盤價算出的報酬，有 2,788 列相差超過 1%，原因待查
TEJ 涵蓋	67,707 筆存續期內的缺漏日尚未逐檔查證；2000-09-04 以前下市的公司不在內；興櫃期間的報價是議價，約 27% 的列沒有成交量；raw 無法重製

表 14 使用建議

需求	建議	要注意的事
長期報酬、技術指標、回測	TEJ 還原價	只有開高低收是還原價；全市場橫斷面要以當日有報價的證券為準
指數策略	TEJ 指數成分 + TEJ 還原價	用 index_members_on() 取當日成分，不要把曾入選的股票放進每一天
全市場選股池、2000 年以前	FinMind 未還原價 + v_securities_all	需自行處理除權息；排除收盤價為 0 的列；無偏誤起點是 2004-02-11
真實成交價、成交量	FinMind 未還原價	2004 年以前的成交量單位不一致
FinMind 還原價	僅用於 2008 年以後、現存公司	結論必須註明存活者偏誤

9 限制與未來工作

目前的限制：

- FinMind 的 token 自 2026-09-19 起降為免費方案，排程全數失敗，資料停在 2026-09-16。失敗紀錄都在 `v_ingestion_runs`。
- TEJ 的每日排程尚未註冊，指數成分以外的股票只靠每週全量更新，最多落後一週。
- TEJ 一側的異常報酬、報酬欄位差異與缺漏日，還沒有逐筆找到解釋。
- 只有日頻價量與指數成分，沒有財報、法人與融資券資料。

接下來的工作：

- 以未還原價與除權息事件自行計算還原因子，補上 FinMind 缺的早期還原與已下市公司，並與 TEJ 對照驗證。
- 把跨來源比對變成每週自動執行的檢查，差異直接進入對帳 view。
- 逐檔查證 TEJ 的缺漏日與異常報酬，必要時回到交易所公告。
- 加入財報與籌碼資料，支援因子研究。

10 結語

這個專案讓我練習了三種能力。

- **資料工程**。分層契約、原子寫入與暫存目錄換檔、跨行程限流、以 COM 自動化一個沒有 API 的來源，以及可以重跑、會自我修復的排程。
- **統計與驗證**。以報酬而非價格水準比較兩個來源，用交易制度做合理性檢驗，把一萬個差異逐筆歸因，並量化存活者偏誤的範圍。
- **研究紀律**。數字一律實測並可以重現；找到自己的錯誤就修正、補上測試並留下變更紀錄；每個結論都寫明適用範圍與尚未驗證的部分。

對財務工程與資料科學而言，模型的可信度不會高於資料的可信度。這個專案的價值不只是一個可以使用的資料庫，也是一套確認資料可信的方法。

附錄 重現方式與檔案位置

本文件的所有數字與圖表都可以由下列程式重新產生。分析程式以唯讀方式連線，不會修改資料庫。

```
cd exports\grad-school-applications
python analysis\01_db_snapshot.py          # 規模與健康狀態 → outputs\tables
python analysis\02_source_comparison.py    # 跨來源比對 → outputs\tables
python build\build_figures.py              # 圖表 → outputs\figures
python build\build_pptx.py                 # 簡報
python build\build_docx.py                 # 本文件
```

表 15 檔案位置

內容	位置
管線程式、設定與文件	data-pipelines\finmind、data-pipelines\tej-wizard
資料與 DuckDB 檔 (不進版本控制)	db\finmind、db\tej-wizard
去重修正的變更紀錄	data-pipelines\finmind\docs\changelog\2026-10-04-adj-dedupe-ticker-first.md
分析程式與結果	exports\grad-school-applications\analysis、outputs\tables
修正前的分析結果	exports\grad-school-applications\outputs\tables_before_fix
互動版資料地圖與 Schema 圖	連結見 data-pipelines\README.md ; Markdown 版為 data-map.md 與 schema-map.md (數字為 2026-09-29 的快照)